

Translation From Natural Language to Ludax through GenLM

Francesco Plastina

francescoplastina@harvard.college.edu

MIT 9.660

Abstract

Systems of rules and rewards guide human behavior. Games, based on sets of rules and rewards, are therefore a perfect scenario to test how humans reason and learn. Game description languages make it possible to specify many board games in a single formalism and run them efficiently for AI or cognitive science experiments. However, these languages are far from the way people naturally talk about games. In this paper, I test whether a language model can translate natural language descriptions of board games into Ludax, a new JAX-based game description language designed for GPU-accelerated simulation (Todd et al., 2025). I focus on a controlled family of two-player, perfect-information “K-in-a-row” grid games that vary board size, winning condition (normal versus misère), diagonal rules, and player asymmetries. Through GenLM, a set of packages for constraining language model generation (Loula et al., 2025), I build a constrained NL to Ludax translation pipeline. First, a JSON schema captures the latent game parameters, then a GPT-2 model fills this schema from natural language, and finally deterministic code maps the JSON into Ludax programs. In order to evaluate whether the translated games are semantically equivalent to their original Ludax counterparts, I use the Intuitive Gamer Model (IGM) (Collins et al., 2025), which simulates human-like play using goal-progress heuristics. For 96 games where IGM could be run on both versions, only 9 produced almost identical outcome metrics. While most translations preserved large-scale properties, systematic shifts in outcome distributions and typical game length were produced, and many distinct descriptions collapsed onto a small set of canonical Ludax games. These results suggest that constrained language model translation can capture some high-level structure of game rules, but often fails to preserve the finer details that shape gameplay.

Keywords: Game Description Languages, Ludax, Intuitive Gamer Model, GenLM

Introduction

Games are a useful testbed for cognitive science because they provide rich, structured environments with well-defined rules and outcomes, yet remain simple enough to experiment with systematically. Recent work has used games both to study human reasoning and to benchmark artificial agents (Collins et al., 2025). At the same time, game description languages like Ludax simplify the process of writing formal specifications of many board games and to compile them to fast simulation code (Todd et al., 2025). However, there is a gap between the formal syntax of a game description language and the way people naturally describe games. For example, a human might say “On a 5x5 grid, the first player needs three in a row to win, but the second player needs only two,” while with Ludax you need a structured description with explicit

parameters, end conditions, and move rules. Bridging this gap has two important consequences. First, it could let casual users describe new games in natural language and automatically obtain runnable environments. Second, it offers a concrete domain in which to study how language models map between everyday descriptions and precise rule systems. In this project, I focus on a restricted but nontrivial class of two-player, perfect-information, place-only games: grid-based “K-in-a-row” games that differ in board size, line length, whether forming a line is good or bad (normal vs. misère play), whether diagonals count, and several simple player asymmetries. For this class of games, both natural language descriptions and Ludax programs are available. My approach can be divided in two steps. First, I use GenLM, a framework for controlling language model outputs (Loula et al., 2025), to build a structured translation pipeline from natural language to Ludax. Rather than asking the model to simply provide a Ludax output, I build a JSON schema that explicitly encodes the latent game parameters. A GPT-2 model is constrained to fill this schema given a description, and then a deterministic program changes the JSON into Ludax. Second, I evaluate the translated games’ semantic equivalence by running the Intuitive Gamer Model (IGM) on both original and translated Ludax codes. The IGM is a cognitively motivated agent that uses fast, “flat” goal-directed simulation to approximate human game play (Collins et al., 2025). Given these double approach, the two main questions of my project are: Can a constrained language model recover the correct latent parameters from short natural language game descriptions? Do the resulting Ludax translations preserve gameplay behavior, as seen through the lens of the Intuitive Gamer Model? The results suggest a mixed answer. Some translations are nearly perfect: the IGM’s outcome distributions, challenge scores, and average game lengths are indistinguishable between the original and translated game. However, such cases are rare. Most translations preserve only a coarse sense of “difficulty” while distorting outcome distributions or typical length, and a substantial subset appears to collapse diverse descriptions into one Ludax games. Every code used for the project is available here saved in two files: Translation.py and Intuitive Gamer Evaluation.py. At the same link it is possible to download the folder with the 121 translations from NL to Ludax.

Related Work

Game description languages and Ludax

Game description languages (GDLs) provide a way to specify large classes of games with a single formalism in order to automatically generate playable environments. Prior work includes Ludii, which can represent over 1,400 board games using a flexible rule description language (Piette et al., 2020), and PGX, a collection of optimized JAX-native implementations of classic games (Koyamada et al., 2024). Ludax builds on these previous ideas: it is a domain-specific language for board games that compiles to JAX code and runs efficiently on GPUs (Todd et al., 2025).

Cognitive models of game play

Collins et al. (2025) have used games as a window into human problem-solving and invention. In particular, they introduce the Intuitive Gamer Model, a simple agent that chooses its moves by combining three aspects: progress toward a goal line, blocking the opponent, and a bias toward central positions (Collins et al., 2025). This model is designed to approximate human behavior in novel games using fast, flat simulations rather than deep tree search for the best moves.

Language models and structured translation

Large language models have always been used to translate between natural language and structured representations such as code or domain-specific languages, but a common problem is ensuring that outputs are syntactically valid and semantically consistent with a target framework. Constrained decoding and the use of intermediate structured forms (like JSON) are basic techniques for making LLMs output respect determined rules. GenLM provides tools for combining language model potentials (how likely each token is) with constraints like JSON schemas and grammars, using sampling methods such as sequential Monte Carlo (Loula et al, 2025). This project applies GenLM to a new task: translating short natural language descriptions of games into a specific Game Description Language (Ludax), and then evaluating the quality of that translation in terms of the resulting gameplay behavior.

Methods

Game set and natural language descriptions

The starting point is a set of 121 natural language descriptions of line-based grid games. Each description has three parts: the specification of width and height, where they are either finite dimensions or infinite, for unbounded boards; a winning condition of the form “K pieces in a row wins” or “loses,” for some integer K; optional clauses describing whether diagonals count (“cannot win by making a diagonal row” or “can only win by making a diagonal row”), asymmetries in required line length (“the first player needs 3 pieces in a row, but the second player only needs 2 pieces”), and first-move advantages (“the first player can place 2 pieces as their first move”). All games used are two-player, perfect-information,

and place-only, which means that players alternate placing pieces on empty cells of a grid until a terminal condition is reached. As correct version of the game’s rules written in Ludax, I use the original 121 games of the Ludax project that I then compare with my translated versions.

JSON schema for latent game parameters

To structure the translation, I use a JSON schema that captures the key latent parameters implied by each description: `board_width`, `board_height`, `win_type` (“win” or “lose”), `p1_winlength`, `p2_winlength`, `p1_diagonals`, `p2_diagonals`, `p1_first_move_pieces`, `p1_first_move_pieces`. This schema is expressive enough to represent normal versus misère play, diagonal restrictions, asymmetric line lengths, and onestep firstmove advantages, while simple enough so that a smaller model (GPT-2) can still produce valid, usable outputs.

NL to JSON with GenLM and GPT-2

I use GenLM’s PromptedLLM, JSON Schema, and Adaptive Weighted Rejection Sampling (AWRS) components to build the NL to JSON mapping. I chose as model a small GPT-2 model (“gpt2”) accessed via GenLM, because of its practicality and previous use with GenLM. I built a short prompt that shows one example description and its corresponding JSON object, both minified to reduce token count. The prompt instructs the model to map short game descriptions to JSON objects with fixed keys and then presents one example description and JSON, followed by a description and JSON for the new input. To establish the constraint, the JSON schema above is compiled into a GenLM potential and combined with the language model using the AWRS sampler. This procedure ensures that the decoded string belongs to the set of valid JSON objects matching the schema. For the sampling, I run sequential Monte Carlo with a small number of particles and a short maximum length (128 tokens), then pick the highest-posterior decoded JSON string and parse it with `json.loads`. This creates a structured representation of the inferred game parameters for each natural language description.

JSON to Ludax compilation

Given a JSON object, I convert it to a lightweight Python dataclass `GameParams`, then to Ludax code.

Board I parse width and height strings; finite numbers are interpreted as integers; infinite number is mapped to a large default size. If width is equal to height, it gives a square board, otherwise a rectangle.

Winning conditions I determine the relevant line length K as $\min(p1_win_length, p2_win_length)$. If `win_type == “win”`, emit an end rule of the form (if (line K) (mover win)); if `win_type == “lose”`, (mover lose). I always include a full board draw rule (if (`full_board`) (draw)). For asymmetries in line length, diagonal rules, or first-move piece counts, I encode the differences in comments rather than fully modeling them in the Ludax rules: this represents a key limitation.

Play rules I use a simple repeated placement rule (repeat (P1 P2) (place (destination empty))), matching the basic “alternate placing pieces” structure of the games. The result is a generated Ludax file for each of the 121 descriptions, saved as *generated_ludax_games/game.i.ltx*.

Intuitive Gamer Model and evaluation metrics

To assess whether the generated Ludax games behave like their original counterparts, I use the Intuitive Gamer Model (IGM) implemented on top of the Ludax environment. For a given Ludax game string, the IGM parses the game into a JAX-based environment (LudaxEnvironment), extracts winning line indices for P1 and P2 using WinConditionExtractor, and detects whether the game is normal or misère, such as whether forming a line is good or bad. Moreover, it defines a policy that, for each legal action in a given state, computes the goal progress: how much the move advances the player toward a completing line; blocking progress: how much it would have advanced the opponent toward a line; a center bias: preference for cells closer to the board center. These components are then combined into logits. It simulates games by sampling actions from the IGM’s logits and for comparison, from a random policy, using the Ludax environment’s transition and reward functions. Following Collins et al. (2025), I summarize each game’s behavior with three features:

EMD feature (emd) negative Earth Mover’s Distance between the IGM-vs-IGM outcome distribution and an “ideal” distribution (more negative means worse).

Challenge feature (challenge) average win rate of IGM against a random policy, across IGM as P1 and IGM as P2 matches.

Length feature (length) mean number of moves in IGM-vs-IGM games. In my implementation, for each valid game I run 120 IGM versus IGM simulations, 240 IGM versus random simulations (half with IGM as P1, half as P2), with fixed random seeds for reproducibility.

Comparing original and generated games

For each game index i , I have an original Ludax file *original_121/game.i.ltx*, a generated Ludax file *generated_ludax_games/game.i.ltx*. For each file, I call the IGM’s evaluate method to obtain emd, challenge, and length. In some cases, evaluation fails due to parsing errors or internal shape mismatches (typically caused by degenerate or unsupported game structures). I only keep games where evaluation succeeds on both versions: these are called the “valid” games. For each valid game, I compute the difference in the three features between the generated ludax games and the original ones. As a simple notion of “near – equivalence,” I define a game as “near equivalent” if $\Delta emd < 0.05$, $\Delta challenge < 0.05$, and $\Delta length < 2$.

Table 1: Comparison metrics between generated and original games

	Δemd	$\Delta challenge$	$\Delta length$
mean	-0.192708	0.020095	7.700174
std	0.352772	0.163418	37.043916
min	-1.000000	-0.108333	-92.158340
25%	-0.389583	-0.084375	-5.1166670
50%	-0.083333	-0.033333	0.254167
75%	0.000000	0.029167	11.664584
max	0.375000	0.750000	117.658333

Results

Evaluation coverage

The Intuitive Gamer was able to evaluate successfully 96 out of 121 games for both original and generated Ludax games. A comparison between the original game and the translated version is shown in the figures below, where I show how game 10 differs in the winning condition: in the original version the player wins with a line of 4 pieces in a row, while in the translated one the player wins with a line of 10 pieces in a row. The remaining 25 games failed the evaluation for at least one version (original or translated), due to either parse errors on some original Ludax files (especially games with complex once-through patterns or infinite boards), or shape mismatches in internal line-index computations for some generated Ludax programs. Therefore, the following quantitative analyses are based on these 96 valid games.

```
(game "game_10"
  (players 2)
  (equipment
    (board (square 10))
  )

  (rules
    (play
      (repeat (P1 P2)
        (place (destination empty))
      )
    )
    (end
      (if (line 4) (mover win))
      (if (full_board) (draw))
    )
  )
)
```

Figure 1: Original game 10

```

1 (game "game_10"
2   (players 2)
3   (equipment
4     (board (square 10))
5   )
6   (rules
7     (play
8       (repeat (P1 P2)
9         (place (destination empty))
10      )
11    )
12  )
13  (end
14    (if (line 10) (mover win))
15    (if (full_board) (draw))
16  )
17 )

```

Figure 2: Translated game 10

Overall differences between original and generated games

Across the 96 valid games, the differences between original and generated metrics were divided as shown in table 1.

diff_emd ($gen_emd - orig_emd$) Looking at table 1, we can see that on average, the generated games are worse with respect to the EMD metric (more negative), but the median difference is modest. A quarter of games have Δemd less than -0.39 , indicating significant degradation in outcome distribution quality, while 75% have Δemd greater than zero, which means most generated games are no worse than the original.

diff_challenge ($gen_challenge - orig_challenge$) Most games show small changes in challenge: the middle 50% lie roughly between -0.08 and $+0.03$. The mean is slightly positive because a few games become much more challenging (up to $+0.75$), but at least half of the games have slightly decreased challenge relative to the original.

diff_length ($gen_length - orig_length$) For this category, we see the greater changes. For a typical game, the average length does not change too much (about 7.7 moves). However, some translations produce extreme changes in game length: in the worst cases, games become roughly more than 100 moves longer than the original, indicating serious changes in pacing and termination conditions. Overall, these data suggest that the translation process often preserves a rough sense of difficulty but can significantly distort outcome distributions and, for a subset of games, drastically alter the expected number of moves.

Near-equivalent games

Using the near-equivalence thresholds defined before ($\Delta emd \leq 0.05$, $\Delta challenge \leq 0.05$, and $\Delta length \leq 2$), I find that 9 out of 96 valid games (about 9.4%) are near-equivalent on all three metrics. For these games, the Intuitive Gamer behaves almost identically in the original and generated environments. Game 1, for example, produced exactly the same self-play outcome distribution, challenge scores, and average length for both versions, down to numerical precision.

Systematic errors and repeated type of games

The IGM output provides a qualitative window into how the translated games behave. Indeed, many of the generated Ludax games share identical or nearly identical IGM statistics, even when their original counterparts show clearly different behavior. For example, the outcome distribution $[0.1583, 0.6250, 0.2167]$ for model-model play, with challenge around 0.91 and average length around 7.84 moves, appears across multiple distinct natural language descriptions. Another recurring pattern, with outcome distribution $[0, 1, 0]$ and challenge 0.95–0.97, is also observed for many translations. These repeated patterns suggest that the NL to Ludax pipeline does not preserve all of the distinctions specified in the text. On the contrary, many descriptions are effectively mapped onto a small set of “canonical” Ludax games that behave similarly under IGM. This is partially because of some asymmetries, like different win lengths for the two players, diagonal-only versus no-diagonals rules, and first-move two-piece advantages, that are currently encoded only in comments rather than in the executable Ludax rules. As a consequence, these details are invisible to the IGM and different descriptions collapse onto the same underlying game dynamics.

Discussion

The research question of the project was: can a constrained language model translate short natural language descriptions of simple games into a formal game description language, preserving gameplay behavior? Using Ludax as the target language (Todd. et al., 2025), GenLM as the translation tool (Loulia et al., 2024), and the Intuitive Gamer Model as a behavioral evaluator (Collins et al., 2025), the answer seems to be that it works sometimes, but not consistently, and only for high-level aspects, not for fine-grained details.

Successes

The key positive result is that some translations are truly successful. For about 9 of the valid games, the generated Ludax produces outcome distributions, challenge levels, and average game lengths that are effectively indistinguishable from the original under the Intuitive Gamer Model. In these cases, the language model faithfully infers board size, win condition, and basic line length from a short natural language description, and the subsequent Ludax code captures enough of the structure to perfectly preserve gameplay. This shows the success of the pipeline: the combination of a JSON schema

and a constrained language model avoids syntax errors and enforces basic type correctness. This is encouraging for future work that aims to let beginners describe games in natural language while still generating valid, runnable environments.

Failures

The broader picture is though less optimistic. Most translations show nontrivial differences between original and generated games: EMD scores are typically worse in the generated games, indicating that their outcome distributions are less aligned with the ideal behavior measured by the original authors of the Intuitive Gamer paper (Collins et al., 2025); challenge scores often drop slightly, suggesting that the generated games are less finely tuned in how they reward or punish the Intuitive Gamer relative to a random agent; game length is highly variable, with some translations making games dramatically longer or shorter. Looking at the qualitative pattern of repeated IGM statistics across many different game descriptions, a deeper issue emerges: semantic collapse. Under the current prompt and model size, GPT-2 tends to fill in the JSON with the same kind of generic, example-like values that pass the schema checks, instead of carefully reflecting about all the specific details mentioned in the game’s description. As a result, the space of generated Ludax games is way smaller than the space of described games. Some of this problem is also due to the Ludax generation code (as predicted before), which encodes asymmetries and diagonal rules only in comments rather than executable rules. Even if the JSON parameters are correct, the resulting Ludax programs do not always reproduce them.

Implications for cognitive science

From a cognitive science perspective, this project sits at the intersection of three themes:

Formalization of games as mental models Ludax provides a compact representation of game rules that can be compiled into executable simulators. Bridging from natural language descriptions to such formal models resembles the process by which people extract structured rules from verbal instructions.

Evaluating models through gameplay behavior Using the Intuitive Gamer Model as a tool to compare original and translated Ludax games changes the question from “Did we parse the text correctly?” to “Does play in this game feel the same?” with the goal of evaluating AI models by their behavior in interactive tasks rather than just static predictions.

Generation and evaluation in design Collins et al. (2025) emphasize that invention requires both generating ideas and evaluating them. Here, the language model plays the role of a generator over game descriptions, while IGM acts as an evaluator. The fact that many translations collapse into a few archetypes suggests that the current generator is not exploring the full design space implied by the natural language prompts.

Limitations

There are several limitations in the project that is worth to note:

Model capacity and prompting I use a small GPT-2 model with a very short few-shot prompt. Larger models with longer prompts may better capture subtle distinctions in the descriptions.

Partial Ludax encoding Some aspects of the JSON schema (for example, player-specific line lengths, diagonal rules, and first-move piece counts) are not fully implemented in the Ludax generator and are instead recorded as comments. This implies semantic mismatch even if the JSON is correct.

Dependence on one behavioral model All evaluation is done through the Intuitive Gamer Model, which is itself an approximation of how human play. It is possible that some differences the IGM is sensitive to would not matter for human players, or vice versa.

Evaluation failures About 25 games could not be evaluated for the generated games, the original ones or both, due to parser or environment issues, which reduces coverage.

Future directions

Based on the previous limitations, the next steps are pretty clear:

Improvement of Ludax generation Implement the full asymmetry and diagonal rules directly in Ludax, rather than in comments. This would reduce sources of mismatch.

Richer supervision and larger models Provide more examples in the prompt, fine-tune a model on pairs of natural language descriptions and JSON schemas, and employing larger language models to better handle the combinatorial variety of possible game descriptions.

Alternative evaluators Compare original and generated games under other agents or, ideally, with human participants.

Extension beyond line-based games The current study focuses on a narrow family of “K-in-a-row” games. Extending to more complex rule systems would better test the limits of the translation between Natural Language and Game Description Language and its relevance for human-like reasoning.

Conclusions

This project demonstrates a first step toward translating natural language game descriptions into a formal game description language (Ludax) using language models and assessing the quality of the translation through gameplay behavior (Intuitive Gamer model). Using GenLM tools, a small language model can often recover the basic structure of simple grid games, and in a minority of cases it can reproduce Ludax programs that are behaviorally indistinguishable from hand-written originals under the Intuitive Gamer Model, starting from natural language descriptions. However, most transla-

tions change the outcome distributions and game length to some degree, and many distinct descriptions are mapped to a bunch of canonical games. For cognitive science, this underlines the current limitations of language models as tools for building formal mental models from informal descriptions. The gap between what is said and what is formally encoded remains substantial. Understanding and reducing that gap is an important direction for future work, both for building better game-playing agents and for modeling how people reason about and invent new rule systems.

Acknowledgments

This project is based on previous work of Katherine Collins and Graham Todd. I particularly want to thank Katherine Collins for her support throughout the ideation and development of the project. Moreover, this work was made possible through the techniques and knowledge built throughout the course of the semester thanks to the always engaging Prof. Tenenbaum's lectures.

References

- Collins, K. M., Todd, G., Zhang, C. E., Weller, A., Togelius, J., Chu, J., ... Tenenbaum, J. B. (2025). Generation and evaluation in the human invention process through the lens of game design. *Proceedings of the Annual Meeting of the Cognitive Science Society*.
- Collins, K. M., Zhang, C. E., Wong, L., Barba da Costa, M., Todd, G., Weller, A., ... Tenenbaum, J. B. (2025). People use fast, flat goal-directed simulation to reason about novel problems. *arXiv preprint arXiv:2510.11503*.
- Koyamada, S., Okano, S., Nishimori, S., Murata, Y., Habara, K., Kita, H., & Ishii, S. (2024). Pgx: Hardware-accelerated parallel game simulators for reinforcement learning. <https://arxiv.org/abs/2303.17503>.
- Loula, J., LeBrun, B., Du, L., Lipkin, B., Pasti, C., Grand, G., ... O'Donnell, T. J. (2025). Syntactic and semantic control of large language models via sequential monte carlo. *International Conference on Learning Representations*.
- Piette, É., Soemers, D. J. N. J., Stephenson, M., Sironi, C. F., Winands, M. H. M., & Browne, C. (2020). Ludii - the ludemic general game system. <https://arxiv.org/abs/1905.05013>.
- Todd, G., Padula, A. G., Soemers, D. J. N. J., & Togelius, J. (2025). Ludax: A gpu-accelerated domain specific language for board games. *arXiv preprint arXiv:2506.22609*.